



## **FACULTY OF ACCOUNTING AND INFORMATICS**

### **DEPARTMENT OF INFORMATION TECHNOLOGY**

#### **INTEGRATIVE PROGRAMMING AND TECHNOLOGIES III**

#### **TEST 2 Memo**

|                      |                           |
|----------------------|---------------------------|
| <b>DATE:</b>         | 3 MAY 2024                |
| <b>TIME:</b>         | 1 ½ Hour                  |
| <b>MARKS:</b>        | 50                        |
| <b>EXAMINER:</b>     | BOAMAH-ABU, CHARLES (PhD) |
| <b>NO. OF PAGES:</b> | 6 (Including cover page)  |

---

#### **INSTRUCTIONS/REQUIREMENTS**

1. This examination is closed book
2. Answer all questions
3. Where stated "briefly", please give a short answer
4. Ensure your handwriting is readable

### Question 1

6 marks

In each of the following, **write only the letter** of your answer.

1. Which of the following does not belong with the others?
  - a. **Interface**
  - b. DAO
  - c. Singleton
  - d. Factory
2. Which of the following does not belong with the others?
  - a. XML
  - b. XSL
  - c. XSLT
  - d. **JSON**
3. Which of the following is not an event in the Java XML stream API?
  - a. Start tag
  - b. End tag
  - c. Characters
  - d. **ATLIST**
4. Write the number of the pattern and the matching description.

|   | Pattern  |   | Description                                                    |
|---|----------|---|----------------------------------------------------------------|
| 1 | Template | A | Allows different patterns to work together                     |
| 2 | Adapter  | B | Allows creation of only one instance of an object              |
|   |          | C | Defines a superclass and allows subclasses to override methods |
|   |          | D | None                                                           |

1 C 2. A

5. Which of the following is a structural pattern?
  - a. Template
  - b. **Adapter**
  - c. Both
  - d. None

### Question 2

8 marks

Figure 1 shows a connection in a client-server application. For each of the following, state whether the statement is **true** or **false** (write **T** for **true** and **F** for **false**).



Figure 1: Client-server application

- 2.1. Both the client and server can read and write
- 2.2. The application will throw an exception if the client is not running and you run the server
- 2.3. The application will throw an exception if the server is not running and you run the client
- 2.4. The connection must specify a port
3. 1. T      2. F      3. T      4. T

### Question 3

4 marks

Briefly answer the following:

1. What is the main difference between **Webservice** and **Microservice** architectures?

2. What is the main disadvantage of RMI?

Answer

1. Any correct answer, e.g., scope of functionalities: Microservices focus on specific business functions (small in scope) while web services are wide in scope
2. Any correct answer, e.g., RMI is language dependent

Question 4

14 marks

The following shows part of an XML document (**postcode.xml**). Each **Area** node contains **code** (i.e., a postcode) and **name** of the area.

```
<PostCode>
  <Area>
    <code>4001</code>
    <name>Durban City</name>
  </Area>
  <Area>
    <code>4092</code>
    <name>Chatsworth</name>
  </Area>
</PostCode>
```

1. Complete the DTD file **postcode.dtd** (below).

6 marks

```
<?xml version="1.0" encoding="UTF-8"?>
```

Answer:

```
<!ELEMENT PostCode (Area)*> ✓✓
<!ELEMENT Area (code,name)*> ✓✓
<!ELEMENT code (#PCDATA)> ✓
<!ELEMENT name (#PCDATA)> ✓
```

2. The following are parts of a program to read the XML in **postcode.xml**. Assume the instance of the XMLStreamReader is **xmlreader**.

2.1. Complete the code segment (below) for a loop to read until end of the XML document. **2 marks**

```
while(_____){
    ...
    ...
}
```

Answer

```
while(xmlreader.hasNext())✓✓){
```

```

...
...
}

```

2.2 The code segment below tests if an event is text, reads the text and stores in a variable called **value**. Assume **event** and **value** have been declared. Complete the code segment. **4 marks**

```

switch(event) {
    ...
    case _____: {
        value= _____;
    }
    break;
    ...
}

```

Answer:

```

switch(event) {
    ...
    ...
    case XMLStreamConstants.CHARACTERS: { //Is it a value in a tag?
        value=xmlreader.getText();
    }
    break;
    ...
}

```

3. Figure 2 shows the resources on a web server. Which of the files should be run to display the XML data in a web page? **2 marks**

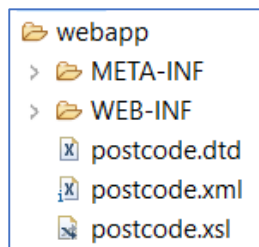


Figure 2: Web resources

Answer: postcode.xml

## Question 5

14 marks

In Figure 3:

- Calculator** is an abstract class; **num** is a number and **calculate()** is an abstract method
- Square** and **SquareRoot** inherit **Calculator**
  - Square** overrides **calculate()** to return the square of a number (**num**).
  - SquareRoot** overrides **calculate()** to return the root of a number (**num**)
- CalculatorFactory** is a factory class. **create(double num, char type)** takes two parameters: **num** (a number) and **type** (a character). If the character is 's', the method will use **num** to create and return an instance of **Square**. If the character is 'r', the method will use **num** to create and return an instance of **SquareRoot**.

- d. Tester creates an instance of **CalculatorFactory** and calls the appropriate method to return an instance of a subclass (i.e, **Square** or **SquareRoot**) of **Calculator**.

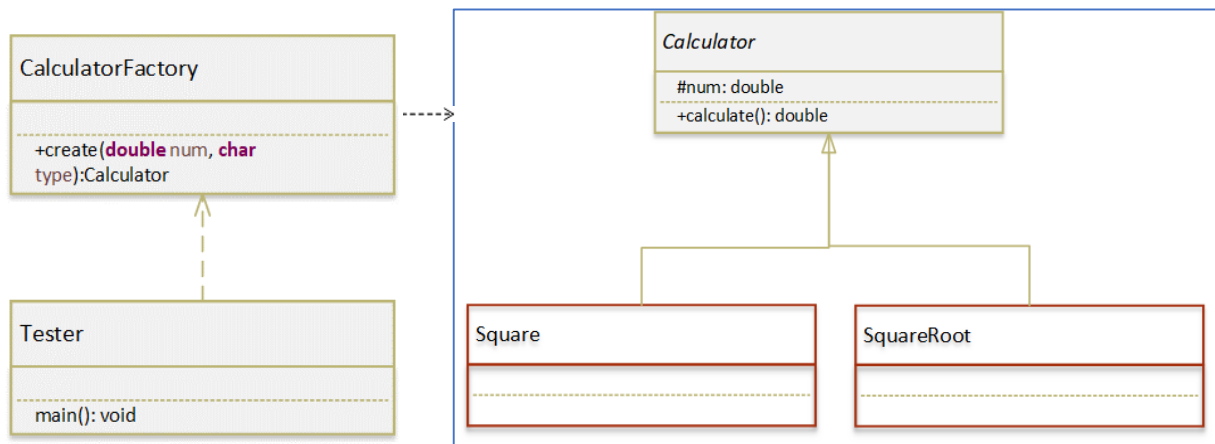


Figure 3: UML diagram

Figure 4 shows the Java classes. Assume **Calculator**, **Square** and **SquareRoot** have been implemented.

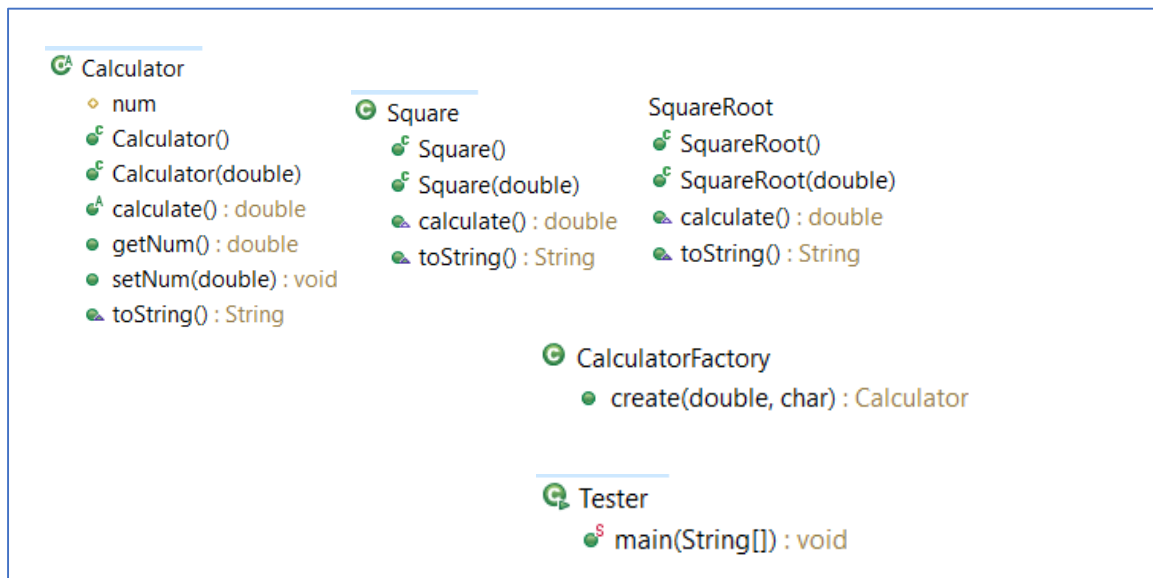


Figure 4: Java classes

1. Complete **create(double num, char type)** in **CalculatorFactory**.

8 marks

```

public class CalculatorFactory {
    _____ create _____{
    _____
    _____
    _____
    _____
    _____
    _____
    _____
    _____
    }
}
  
```

Answer

```

package patternfactory;
public class CalculatorFactory {
    public Calculator ✓create(double num, char type) ✓✓{
        Calculator c=null;
        ✓✓ if(type=='s') {
            c=new Square(num);
        }
        ✓✓ if(type=='r') {
            c=new SquareRoot(num);
        }
        ✓ return c;
    }
}
  
```

2. Complete **main()** in Tester as follows:

- Create an instance of **CalculatorFactory** called **cf**
- Use **cf** to call a method to return the appropriate instance of a subclass and display on the screen. Use the values below.
  - a. 10 and 's'
  - b. 16 and 'r'

A typical output is shown below.

6 marks

```
Number is 10.0
Square is 100.0

Number is 16.0
Square root is 4.0
```

```
package patternfactory;
public class Tester {
    public static void main(String[] args) {
        _____
        _____
        _____
    }
}
```

Answer

```
package patternfactory;
public class Tester {
    public static void main(String[] args) {
        ✓✓ CalculatorFactory cf=new CalculatorFactory();
        ✓✓ System.out.println(cf.create(10, 's'));
        ✓✓ System.out.println(cf.create(16, 'r'));
    }
}
```

### Question 6

8 marks

The server-side of a client-server application writes a number into a stream, and the client-side reads the number from the stream. The server is **127.0.0.1** and the connection is on port **1999**.

Figure 5 shows the class **Client** (part of the client-side application).

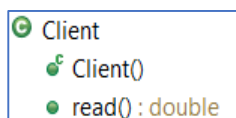


Figure 5: The class **Client** (part of the client-side application)

The method **read()**:

- Sets up the appropriate socket and stream
- Reads a number from the stream
- Closes the stream and the socket

- d. Returns the number
- e. Handles all possible exceptions.

Complete **read()** below.

```
public double read() throws Exception {
```

```
_____  
_____  
_____  
_____  
_____  
_____
```

```
}
```

Answer

```
public double read() throws Exception{
```

```
    Socket s=new Socket("127.0.0.1", 1999); ✓✓
```

```
    InputStream is=s.getInputStream();✓
```

```
    DataInputStream dis=new DataInputStream(is); ✓
```

```
    double ans=dis.readDouble(); ✓✓
```

```
    s.close();✓
```

```
    return ans; ✓
```

```
}
```